universität innsbruck

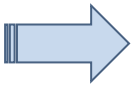
Operatoren und Ausdrücke

Einführung in die Programmierung

Michael Tschuggnall
Universität Innsbruck

Überblick

- Einführung
- Vorschau
- Variablen und Datentypen
- Operatoren und Ausdrücke
- Kontrollstrukturen
- Funktionen
- Arrays
- Zeiger
- Strukturen, Unions, Bitfelder
- Speicherklassen
- Dynamische Speicherverwaltung
- Präprozessor
- Modulare Programmierung
- Ein- und Ausgabe
- gcc und make



- Terminologie
- Operatoren
- Typumwandlung
- Zusätzliche Übungsaufgaben

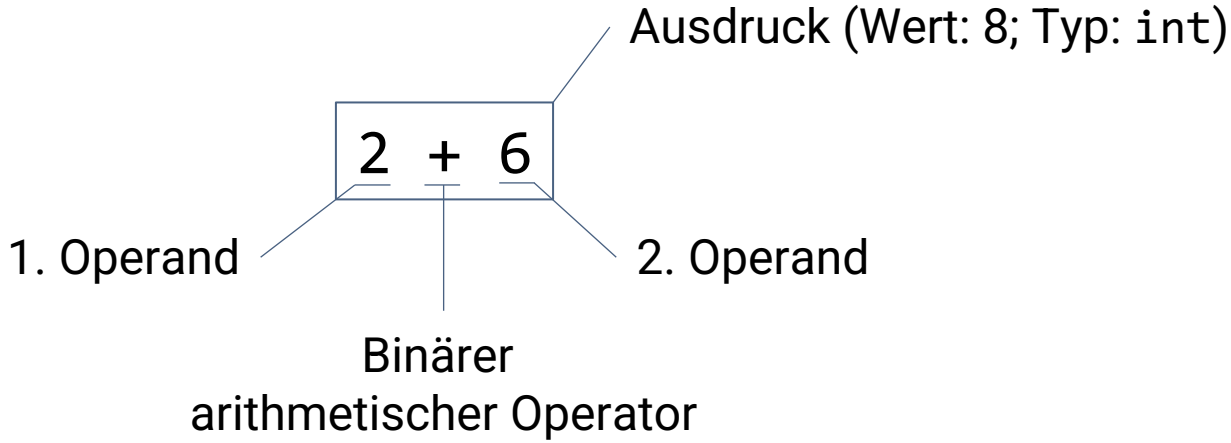
Terminologie

Operatoren

- Mithilfe von Operatoren können beispielsweise Variablen mit Inhalten belegt, oder die in ihnen gespeicherten Werte verändert werden.
- Einteilung basierend auf der Anzahl der Operanden:
  - Unäre Operatoren (besitzen einen Operanden)
  - Binäre Operatoren (besitzen zwei Operanden)
  - Ternäre Operatoren (besitzen drei Operanden)
- Einteilung basierend auf der Position:
  - Präfix-Form: Operator steht vor seinem bzw. seinen Operanden.
  - Postfix-Form: Operator steht hinter seinem bzw. seinen Operanden.
  - Infix-Form: Operator steht zwischen seinen Operanden (binäre Operatoren).
- Einteilung basierend auf der Assoziativität:
  - Linksassoziativ: Auswertung von links nach rechts.
  - Rechtsassoziativ: Auswertung von rechts nach links.

Ausdruck (expression)

- Ein Ausdruck ist eine Folge von Literalen, Bezeichnern und Operatoren.
- Ein Ausdruck hat einen Typ und einen Wert.
- Ausdrücke können Seiteneffekte aufweisen.
- Einfache Ausdrücke sind:
  - Literale
  - Variablen
  - Funktionen
- Durch das Verknüpfen von Operanden mit Operatoren können komplexe Ausdrücke entstehen.



Anweisung (statement)

- Durch Anhängen eines Semikolons wird aus einem Ausdruck eine Anweisung (Ausdrucksanweisung).
- Eine Anweisung spezifiziert eine Aktion, welche durchgeführt werden soll.
- Die Anweisungen eines Programms bestimmen die Funktionalität.
- Anweisungen können keinen Wert haben.
  - Solche Anweisungen können nicht Teil eines größeren Ausdrucks sein.
- Beispiel:
  - `2 + 6;`
    - Möglich, aber nicht sinnvoll.
    - Sollte zugewiesen werden.
  - `i++;`
    - Möglich und sinnvoll, da ++ einen Seiteneffekt hat.
    - i wird um 1 erhöht!

Seiteneffekte

- Bei einem Seiteneffekt wird ein Zustand verändert.
- Beispiele:
  - Der Wert einer Variable.
  - Der Inhalt einer Datei.
  - Ausgabe auf der Standardausgabe.
- Operatoren können Seiteneffekte haben.
- Seiteneffekte werden auch als Nebeneffekte bezeichnet.

Operatoren

Zuweisungsoperator

- Der Zuweisungsoperator (=) ist ein binärer Operator.
- Bei einer Zuweisung wird der Wert des rechten Operanden dem Speicherplatz auf der linken Seite zugewiesen (Seiteneffekt).
- Ein Zuweisungsausdruck hat den Wert des linken Operanden nach der Zuweisung und den Typ des linken Operanden.
  - Zuweisungen können in Ausdrücken weiterverwendet werden.
- Der Zuweisungsoperator arbeitet von rechts nach links (rechtsassoziativ).
- Die Anweisung `x = x + 1;` macht Sinn!
  - Der Wert aus x wird um 1 erhöht, der neue Wert wird in x gespeichert.

Arithmetische Operatoren

- Arithmetische Operatoren werden für Berechnungen verwendet
  - Unäre arithmetische Operatoren (+ , -) sind rechtsassoziativ.
  - Binäre arithmetische Operatoren (+ , - , * , /) sind linksassoziativ.
- Für die Division und den Restwert ist das Ergebnis bei Ganzzahlen undefiniert, wenn der zweite Operand 0 ist.

Operator	Name	Beispiel	Operandentyp	Operator liefert
-	Minus (unär)	`-x`	Arithmetische Typen	Vorzeichenwechsel
+	Plus	`x + y`	Arithmetische Typen	Summe
-	Minus	`x - y`	Arithmetische Typen	Differenz
*	Multiplikation	`x * y`	Arithmetische Typen	Produkt
/	Division	`x / y`	Arithmetische Typen	Ganzzahlige Division bei ganzen Zahlen (Nachkommaanteil wird abgeschnitten), Quotient
%	Restwert	`x % y`	Ganzzahl-Typen	Rest bei ganzzahliger Division

Beispiel 1 - Arithmetische Operatoren

example_arithmetic_operators1.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("1. %d\n", 4 - 2);
    printf("2. %d\n", 5 + 2 + 1);
    printf("3. %d\n", 4 * 0);
    printf("4. %f\n", 8.5 * 2);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
1. 2
2. 8
3. 0
4. 17.000000
```

Beispiel 2 - Arithmetische Operatoren

example_arithmetic_operators2.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("1. %d\n", 5 / 2);
    printf("2. %f\n", 5.0 / 2.0);
    printf("3. %d\n", -5 / 2);
    printf("4. %f\n", -5 / 2.0);
    printf("5. %d\n", 5 % 2);
    printf("6. %d\n", -5 % 2);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
1. 2
2. 2.500000
3. -2
4. -2.500000
5. 1
6. -1
```

Überlauf bei Ganzzahlen (1)

- Ein Überlauf bei Ganzzahlen (integer overflow) tritt auf, wenn das Ergebnis einer arithmetischen Operation nicht in den Wertebereich des Datentyps passt.
- Bei vorzeichenbehafteten Ganzzahlen (`signed`) ist das Verhalten bei einem Überlauf undefiniert!
- Bei vorzeichenlosen Ganzzahlen (`unsigned`) wird nach jeder arithmetischen Operation modulo 2^n gerechnet, wobei n die Anzahl der Bits des Datentyps ist.

Überlauf bei Ganzzahlen (2)

example_overflow.c

```c
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    unsigned a = UINT_MAX;
    unsigned b = 0;
    printf("%u\n", a);
    printf("%u\n", b);
    printf("%u\n", a + 1);
    printf("%u\n", b - 1);
    return EXIT_SUCCESS;
}
```

Maximaler Wert von `unsigned` aus `limits.h` (plattformabhängig!)

Ausgabe auf der Standardausgabe (plattformabhängig):

```
4294967295
0
0
4294967295
```

Inkrement- und Dekrementoperatoren (1)

- Der Inkrementoperator (++) bzw. der Dekrementoperator (--) kann als Prä- oder Postfix-Operator verwendet werden.
- Diese Operatoren weisen einen Nebeneffekt auf:
  - Inkrementoperator: Wert der Variable wird um 1 erhöht (inkrementiert).
  - Dekrementoperator: Wert der Variable wird um 1 verringert (dekrementiert).

Operator	Benennung	Beispiel	Nebeneffekt	Wert des Ausdrucks
`++`	Präinkrement	`++x`	`x = x + 1`	Wert von x nach dem Inkrement
`++`	Postinkrement	`x++`	`x = x + 1`	Wert von x vor dem Inkrement
`--`	Prädekrement	`--x`	`x = x - 1`	Wert von x nach dem Dekrement
`--`	Postdekrement	`x--`	`x = x - 1`	Wert von x vor dem Dekrement

Inkrement- und Dekrementoperatoren (2)

example_increment_decrement.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
   int a = 3;
   int b = 3 * ++a;
   int c = 3 * a++;
   printf("a = %d\n", a);
   printf("b = %d\n", b);
   printf("c = %d\n", c);
   return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
a = 5
b = 12
c = 12
```

Aufgabe Inkrement- und Dekrementoperatoren

- Was gibt das folgende Programm auf der Standardausgabe aus?

exercise_increment_decrement.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i = 1;
    printf("%d\n", i--);
    printf("%d\n", ++i);
    printf("%d\n", 2 * i++);
    printf("%d\n", 2 * --i);
    return EXIT_SUCCESS;
}
```

Aufgabe Inkrement- und Dekrementoperatoren - Lösung

- Was gibt das folgende Programm auf der Standardausgabe aus?

exercise_increment_decrement.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i = 1;
    printf("%d\n", i--);
    printf("%d\n", ++i);
    printf("%d\n", 2 * i++);
    printf("%d\n", 2 * --i);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
1
1
2
2
```

Vergleichsoperatoren

- Vergleichsoperatoren vergleichen zwei Operanden und liefern einen Wert vom Typ `int`.
- Der Wert eines Vergleichs ist 0, wenn der entsprechende Vergleich falsch ist, andernfalls ist das Ergebnis 1.
- Vergleichsoperatoren sind linksassoziativ.
- Vergleichsoperatoren:

C - Notation	Math. Notation
`a < b`	a < b
`a > b`	a > b
`a <= b`	a ≤ b
`a >= b`	a ≥ b
`a == b`	a = b
`a != b`	a ≠ b

Logische Operatoren

- Mit Hilfe logischer Operatoren können Ausdrücke als Aussagen miteinander verknüpft werden.
  - Der entstehende Ausdruck hat Typ `int` und den Wert 1, wenn die Aussage wahr ist, ansonsten hat er den Wert 0.
  - Jeder Operand mit dem Wert 0 wird als falsch interpretiert. Jeder Wert, der nicht 0 ist, wird als wahr angesehen.
- Die Operatoren && und || werden von links nach rechts mittels Kurzschlussauswertung (**short circuit evaluation**) ausgewertet.
  - Die Auswertung eines Ausdrucks bricht ab, sobald der endgültige Wert dieses Ausdrucks unabänderlich feststeht.
- Logische Operatoren sind linksassoziativ.

Operator	Name	Beispiel	Resultat
&&	Logisches UND	`x && y`	1, wenn x und y ungleich 0 sind; sonst 0
||	Logisches ODER	`x || y`	1, wenn x oder y ungleich 0 sind; sonst 0
!	Logische Negation	`!x`	1, wenn x gleich 0 ist; sonst 0

Beispiel logische Operatoren (1)

- Gegeben seien die folgenden Deklarationen:

```
1 int x = 5;
2 int y = 9;
3 int z = 0;
```

Ausdruck	Wert
`x && y`	1
`!x`	0
`!(y - (x + 4))`	1
`z && (x + y)`	0
`(z < x) && (x < y)`	1
`(x == 3) || (y == 9)`	1

Beispiel logische Operatoren (2)

example_logical_operators.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 0;
    int b = 0;
    printf("%d\n", (a++) && (++b));
    printf("a = %d\n", a);
    printf("b = %d\n", b);
    return EXIT_SUCCESS;
}
```

Dient nur für die Veranschaulichung der Kurzschlussevaluierung. Seiteneffekte beim 2. Operanden bei && bzw. || sollten vermieden werden!

Ausgabe auf der Standardausgabe:

```
0
a = 1
b = 0
```

Aufgabe logische Operatoren

- Was gibt das folgende Programm auf der Standardausgabe aus?

exercise_logical_operators.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 1;
    int b = 2;
    printf("%d\n", !(a <= b));
    printf("%d\n", !(a - b + 1));
    printf("%d\n", (a < b) && (b >= 2));
    printf("%d\n", (a <= 0) && (b = 0));
    printf("%d\n", (a = 5) || (b == 0));
    printf("%d\n", b || (a != 5));
    return EXIT_SUCCESS;
}
```

Aufgabe logische Operatoren - Lösung

- Was gibt das folgende Programm auf der Standardausgabe aus?

exercise_logical_operators.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 1;
    int b = 2;
    printf("%d\n", !(a <= b));
    printf("%d\n", !(a - b + 1));
    printf("%d\n", (a < b) && (b >= 2));
    printf("%d\n", (a <= 0) && (b = 0));
    printf("%d\n", (a = 5) || (b == 0));
    printf("%d\n", b || (a != 5));
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
0
1
1
0
1
1
```

Bitoperationen

- Mit diesen Operatoren wird ein direkter Zugriff auf die binäre Darstellung von Ganzzahlen unterstützt.
- Mithilfe der Operatoren in der folgenden Tabelle können boolesche Operationen auf jede Bitposition angewendet werden.

Operator	Name	Beispiel	Resultat pro Bitposition
&	Bitweises UND	x & y	1, wenn x und y an dieser Position 1 sind; sonst 0
|	Bitweises ODER	x | y	1, wenn x oder y an dieser Position 1 sind; sonst 0
^	Bitweises Exklusives-ODER	x ^ y	1, wenn x oder y 1 sind, aber nicht beide; sonst 0
~	Bitweise Negation	~x	1, wenn x an dieser Position 0 ist; sonst 0

- Durch Shift-Operationen kann das Bit-Muster verschoben werden.

Operator	Name	Beispiel	Resultat
<<	Linksshift	x << y	Jeder Bitwert in x wird um y Positionen nach links verschoben.
>>	Rechtsshift	x >> y	Jeder Bitwert in x wird um y Positionen nach rechts verschoben.

Beispiel Bitoperationen

- Gegeben seien die folgenden Deklarationen:

```
1 unsigned x = 5u;
2 unsigned y = 9u;
```

Ausdruck	Binärwert	Dezimalwert
x	101	5
y	1001	9
x & y	1	1
x | y	1101	13
x ^ y	1100	12
x << 1u	1010	10
x >> 2u	1	1

Aufgabe Bitoperationen

- Was gibt das folgende Programm auf der Standardausgabe aus?

exercise_bit_operations.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    unsigned a = 32u;
    unsigned b = 15u;
    unsigned c = 17u;
    printf("1. %u\n", a | 1u);
    printf("2. %u\n", b & 0u);
    printf("3. %u\n", c ^ 3u);
    printf("4. %u\n", a << 2u);
    printf("5. %u\n", a >> 1u);
    printf("6. %u\n", a >> 4u);
    return EXIT_SUCCESS;
}
```

Aufgabe Bitoperationen - Lösung

- Was gibt das folgende Programm auf der Standardausgabe aus?

exercise_bit_operations.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    unsigned a = 32u; // 32 in binary: 100000
    unsigned b = 15u; // 15 in binary:   1111
    unsigned c = 17u; // 17 in binary:  10001
    printf("1. %u\n", a | 1u);
    printf("2. %u\n", b & 0u);
    printf("3. %u\n", c ^ 3u);
    printf("4. %u\n", a << 2u);
    printf("5. %u\n", a >> 1u);
    printf("6. %u\n", a >> 4u);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
1. 33
2. 0
3. 18
4. 128
5. 16
6. 2
```

Kombinierte Zuweisungsoperatoren

- Durch die kombinierten Zuweisungsoperatoren können arithmetische Operationen oder Bitoperationen zusammen mit einer Zuweisung durchgeführt werden.
- Operatoren:
  - `+=, -=, *=, /=, %=` (Arithmetische Operationen mit Zuweisung)
  - `&=, |=, ^=, <<=, >>=` (Bitoperationen mit Zuweisung)
- Beispiele:

Kombinierte Zuweisung	Äquivalente Anweisung
`a += 2;`	`a = a + 2;`
`b <<= 1;`	`b = b << 1;`
`c *= a - 10;`	`c = c * (a - 10);`

Aufgabe Zuweisungsoperatoren

- Was gibt das folgende Programm auf der Standardausgabe aus?

exercise_compound_assignments.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 5;
    int b = 4;
    int c = 9;
    int d = 7;
    a += b;
    b -= (c % b);
    c *= (d + 3);
    d /= 3;
    printf("a = %d, b = %d, c = %d, d = %d\n", a, b, c, d);
    return EXIT_SUCCESS;
}
```

Aufgabe Zuweisungsoperatoren - Lösung

- Was gibt das folgende Programm auf der Standardausgabe aus?

exercise_compound_assignments.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int a = 5;
    int b = 4;
    int c = 9;
    int d = 7;
    a += b;
    b -= (c % b);
    c *= (d + 3);
    d /= 3;
    printf("a = %d, b = %d, c = %d, d = %d\n", a, b, c, d);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
a = 9, b = 3, c = 90, d = 2
```

Bedingungsoperator

- In C gibt es nur einen ternären Operator, den sogenannten Bedingungsoperator.
- Dieser rechtsassoziative Operator hat die Form:
  - `A ? B : C`
  - Beim Bedingungsoperator wird zuerst A ausgewertet.
  - Ist der Wert von A ungleich 0, wird B ausgewertet und stellt das Ergebnis für den Ausdruck dar, ansonsten C.
- Beispiel:

```
max_ab = (a > b) ? a : b;
```

  - `max_ab` erhält das Maximum der Variablen `a` und `b`.

sizeof-Operator (1)

- Der Operator **`sizeof`** dient zur Ermittlung der Größe von Datentypen und Datenobjekten im Hauptspeicher in Bytes.
- Der Operator kann sowohl auf einen Ausdruck als auch auf einen Typbezeichner angewendet werden.
  - Ein Ausdruck wird nicht ausgewertet, es wird nur der Typ und dessen Größe bestimmt.
- `sizeof` liefert ein Ergebnis vom Typ `size_t`!
  - Der Typ `size_t` ist eine vorzeichenlose Ganzzahl (definiert in `stddef.h`).
  - Architekturabhängig (32-Bit, 64-Bit)!
- Beispiele:
  - `sizeof(i)`
  - `sizeof i`
  - `sizeof(int)`
  - `sizeof(3 + 1)`
  - `sizeof(3 / 0)`

sizeof-Operator (2)

- Klammerung:
  - Klammern nach `sizeof` können bei einem Ausdruck weggelassen werden.
    - Sie werden aber üblicherweise gesetzt.
  - ACHTUNG: `sizeof` hat hohe Präzedenz!
    - `sizeof x + y` wird vom Compiler akzeptiert, ermittelt aber zuerst die Größe von x und zählt dann y dazu!
    - Besser: `sizeof(x) + y`
  - Bei einem Datentyp müssen die Klammern gesetzt werden.

Operatorpräzedenz und Assoziativität (1)

- Wie in der Mathematik spielt es auch in C eine wichtige Rolle, welche Rangfolge und Assoziativität Operatoren haben.
  - Beispiel: `5 + 2 * 3` ist äquivalent zu `5 + (2 * 3)` und ergibt `11`.
- Operatoren haben unterschiedliche **Prioritäten** (**Präzedenz**):
  - Ähnlich zu „Punktrechnung vor Strichrechnung“.
  - Regelt die Rangfolge bei Operatoren auf verschiedenen Stufen.
- **Assoziativität:**
  - Regelt die Rangfolge bei Operatoren auf gleicher Stufe.
- Durch das explizite Setzen von Klammern kann die Rangfolge geändert werden.
  - Beispiel: `(5 + 2) * 3` hat den Wert `21`.

Operatorpräzedenz und Assoziativität (2)

Operatoren (höchste bis niedrigste Priorität)	Assoziativität
Postfix Operatoren: `() [] {} -> . ++ --`	links
Präfix Operatoren: `! ~ ++ -- + - * & sizeof (type) _Alignof`	rechts
Multiplikative Operatoren: `* / %`	links
Additive Operatoren: `+ -`	links
Shift Operatoren: `<< >>`	links
Vergleichsoperatoren: `< <= > >=`	links
Gleichheitsoperatoren: `== !=`	links
Bitweises UND: `&`	links
Bitweises XOR: `^`	links
Bitweises ODER: `|`	links
Logisches UND: `&&`	links
Logisches ODER: `||`	links
Bedingungsoperator: `?:`	rechts
Zuweisungsoperatoren: `= += -= *= /= %= &= ^= |= <<= >>=`	rechts
Komma-Operator: `,`	links

Operatorpräzedenz und Assoziativität (3)

- Beispiele:

Ausdruck	Äquivalenter Ausdruck mit expliziter Klammerung
`a && b || c < 0`	`(a && b) || (c < 0)`
`6 * 5 + 4 << 3 / 2`	`((6 * 5) + 4) << (3 / 2)`
`a < 5 == 0`	`(a < 5) == 0`
`a / b % c`	`(a / b) % c`
`1 + 2 + 3 + 4 * 5`	`((1 + 2) + 3) + (4 * 5)`

Auswertungsreihenfolge

- **Achtung**: Die Reihenfolge der Verknüpfung hat nichts mit der Reihenfolge der Auswertung der Operanden zu tun!
- 1 * 2 + 3 * 4 hat ein eindeutiges Ergebnis, jedoch wird durch den Standard nicht vorgegeben, ob zuerst der Teilausdruck 1 * 2 oder 3 * 4 ausgewertet wird.

Typumwandlung

Typumwandlung

- Bei der Umwandlung von einem Datentyp in einen anderen wird in C ein sogenanntes Type-Casting durchgeführt.
- Es gibt zwei Möglichkeiten, den Datentyp zu ändern:
  - Explizite Typumwandlung:
    - Die Konvertierung des Datentyps wird erzwungen.
  - Implizite Typumwandlung:
    - Automatische Konvertierung von einem zum anderen Datentyp durch den Compiler.
    - Dies geschieht, wenn z. B. einer Variable vom Typ `float` ein `int`-Wert zugewiesen wird.
- Bei expliziten und impliziten Typumwandlungen:
  - Wird vorausgesetzt, dass der Compiler eine Typumwandlung auch wirklich unterstützt.
  - Gelten dieselben Konvertierungsregeln.

Explizite Typumwandlung

- Explizite Typumwandlung kann durch den Typkonvertierungsoperator (cast-Operator) durchgeführt werden.
- Der cast-Operator ist ein unärer Operator.
- Allgemeine Form des cast-Operators: `(Typname) Ausdruck`

example_cast.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("%d\n", (int) 5.6);
    printf("%f\n", (double) (4 / 3));
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
5
1.000000
```

Implizite Typumwandlung

- Es ist nicht notwendig, dass die Operanden eines Ausdrucks vom selben Typ sind.
- Auch bei einer Zuweisung muss der Typ der Operanden nicht übereinstimmen.
- In solchen Fällen (und noch anderen, die später besprochen werden) kann der Compiler implizite Typumwandlungen durchführen.
  - Diese Umwandlungen laufen nach bestimmten Regeln ab.
  - Diese Umwandlungen erfolgen nur zwischen verträglichen Typen.

Typumwandlung – _Bool

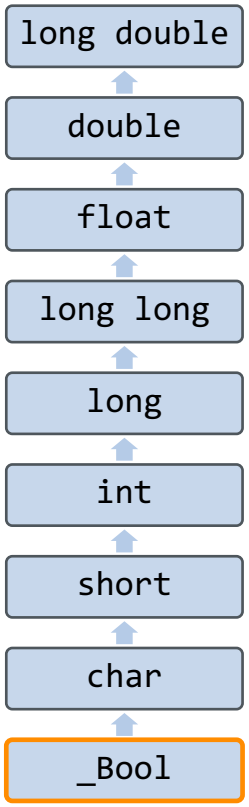
- Beliebiger arithmetischer Typ → _Bool:
  - Jeder skalare Wert, der bei einem Vergleich gleich null ist (== 0) ergibt den Wert 0, andernfalls ist der Wert 1.
  - Beispiel:

```
int i1 = 123;
int i2 = -7;
int i3 = 0;

_Bool b1 = i1;        // implizite Umwandlung
_Bool b2 = i2;        // implizite Umwandlung
_Bool b3 = i3;        // implizite Umwandlung
```

→ b1 und b2 haben den Wert 1, b3 hat den Wert 0

- _Bool → beliebiger arithmetischer Typ:
  - Darstellung des _Bool Werts (0 bzw. 1) in der Repräsentation des entsprechenden Typs.



Typumwandlung – Ganzzahl-Typen (außer _Bool)

- Ganzzahl-Typ → vorzeichenloser Ganzzahl-Typ:
  - Es wird wiederholt der Maximalwert des neuen Typs +1 addiert bzw. subtrahiert, bis der Wert im Wertebereich des neuen Typs liegt (entspricht Modulo)
  - Das kann zum Verlust der Größenordnung führen!
  - Beispiel 1:

```
int a = -1;
unsigned int b = a;       // implizite Umwandlung
```

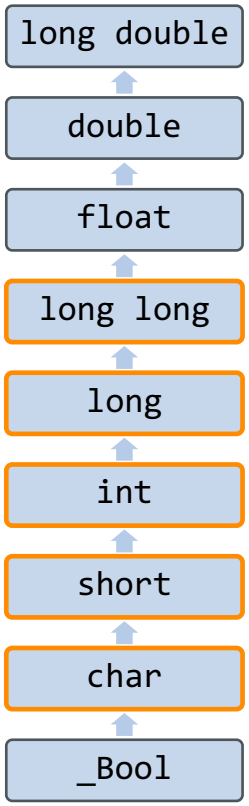
→ b hat den Wert -1 + (UINT_MAX + 1) = 4294967295 (abh. von UINT_MAX)

  - Beispiel 2:

```
long x = 1025;
unsigned char y = (unsigned char)x;  // expliziter Cast
```

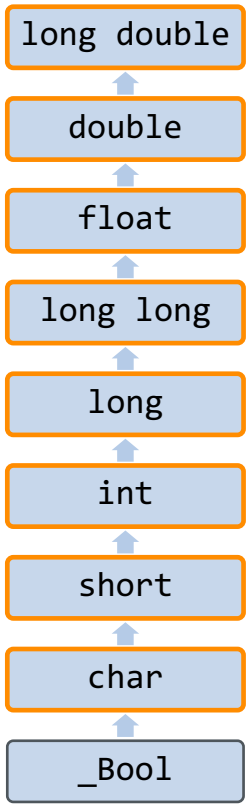
→ y hat den Wert 1025 – 256 – 256 – 256 – 256 = 1

- Ganzzahl-Typ → vorzeichenbehafteter Ganzzahl-Typ:
  - Der Wert bleibt unverändert, sofern der Wert im Wertebereich des neuen Typs liegt.
  - Kann der Wert im neuen Typ nicht abgebildet werden, ist das Verhalten implementierungsabhängig.



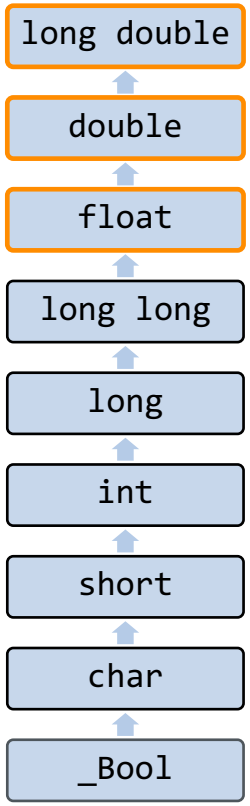
Typumwandlung – Gleitkomma- und Ganzzahl-Typen

- Gleitkomma-Typ → Ganzzahl-Typ (außer _Bool):
  - Nachkommastellen werden abgeschnitten (runden Richtung 0).
  - Liegt der Vorkommanteil innerhalb des Wertebereichs des neuen Typs, wird dieser Wert exakt dargestellt.
  - Andernfalls liegt undefiniertes Verhalten vor.
- Ganzzahl-Typ → Gleitkomma-Typ:
  - Der Wert wird exakt dargestellt, sofern dies in der Gleitkommarepräsentation möglich ist.
  - Liegt der Wert innerhalb des darstellbaren Bereichs, kann aber nicht exakt dargestellt werden, so wird (implementierungsabhängig) der nächste darüber bzw. darunterliegende darstellbare Wert verwendet.
  - Liegt der Wert außerhalb des darstellbaren Bereichs, liegt undefiniertes Verhalten vor.



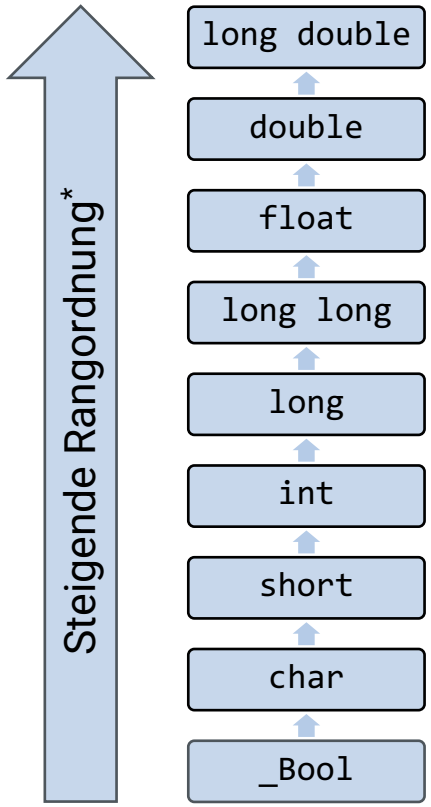
Typumwandlung – Gleitkomma-Typen

- Gleitkomma-Typ → Gleitkomma-Typ:
  - Der Wert wird exakt dargestellt, sofern dies in der Repräsentation des neuen Typs möglich ist.
  - Liegt der Wert innerhalb des darstellbaren Bereichs, kann aber nicht exakt dargestellt werden, so wird (implementierungsabhängig) der nächste darüber bzw. darunterliegende darstellbare Wert verwendet.
  - Liegt der Wert außerhalb des darstellbaren Bereichs, liegt undefiniertes Verhalten vor.



Gewöhnliche arithmetische Typumwandlung

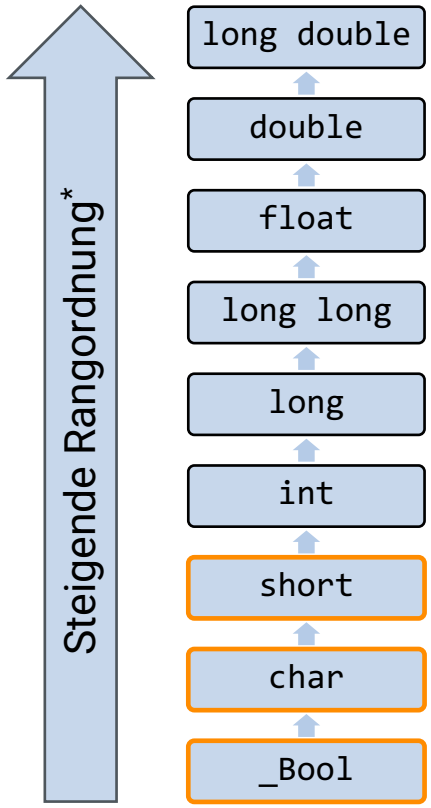
- Wird bei allen binären Operatoren außer logischem UND (&&), logischem ODER (||) und den Zuweisungsoperatoren durchgeführt.
- Wird auch beim Bedingungsoperator (?) durchgeführt, um den Typ des Ergebnisses zu ermitteln.
- Regeln:
  1. Finden des Typs mit höherem Rang.
  2. Ist der Typ mit höherem Rang ein Gleitkomma-Typ:
     - Umwandung des Typs mit niedrigerem Rang in den Typ mit höherem Rang.
  3. Sind beide Typen Ganzzahl-Typen wird zunächst Integer Promotion (siehe nächste Folie) und anschließend die Umwandlung der Ganzzahlen durchgeführt.



* `signed` und `unsigned` Varianten desselben Ganzzahl-Typs haben dieselbe Rangordnung.

Integer Promotion

- Ist eine implizite Typumwandlung auf `int` bzw. `unsigned int`, die für Ganzzahl-Typen mit geringerem Rang als dem Rang von `int` infrage kommt.
  - Sofern alle Werte des Ausgangstyps in einem `int` abgebildet werden können, wird `int` verwendet, sonst `unsigned int`.
  - Ändert den repräsentierten Wert und das Vorzeichen nicht.
- Falls der Rang des Ausgangstyps geringer als der Rang von `int` und `unsigned int` ist, wird Integer Promotion angewendet bei:
  - Gewöhnlichen arithmetischen Typumwandlungen
  - Den unären +, - und ~ Operatoren
  - Beiden Operanden der Shift Operatoren.



* `signed` und `unsigned` Varianten desselben Ganzzahl-Typs haben dieselbe Rangordnung.

Gewöhnliche arithmetische Typumwandlung – Umwandlung bei Ganzzahl-Typen (1)

1. Sind nach der Integer Promotion beide Typen gleich, ist keine weitere Umwandlung nötig.
   - Beispiel:

     ```
     signed char a = 5;
     short b = 3;
     ```

     Ausdruck: `a + b` → sowohl `a` als auch `b` werden zu `int` promotet

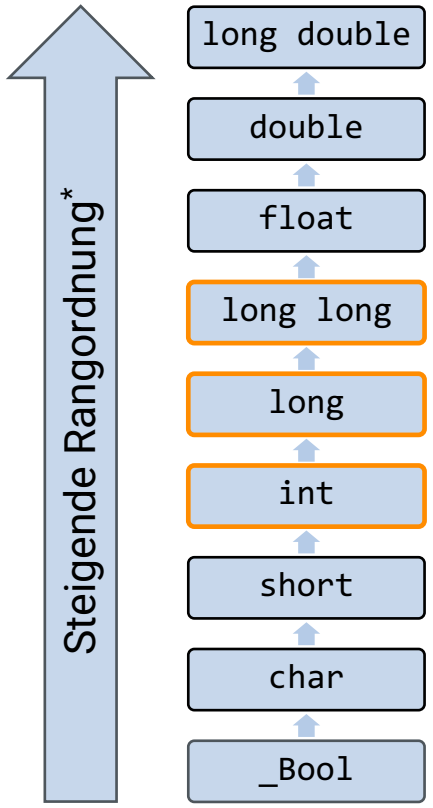
2. Sind beide Typen `signed` oder `unsigned`, wird der Typ mit niedrigerem Rang in den Typ mit höherem Rang umgewandelt.
   - Beispiel:

     ```
     int a = 1000;
     long b = 2000L;
     ```

     Ausdruck: `a + b` → `a` wird zu `long` umgewandelt

     ```
     unsigned c = 100u;
     unsigned long long d = 200ull;
     ```

     Ausdruck: `c + d`
     → `c` wird zu `unsigned long long` umgewandelt



* `signed` und `unsigned` Varianten desselben Ganzzahl-Typs haben dieselbe Rangordnung.

Gewöhnliche arithmetische Typumwandlung – Umwandlung bei Ganzzahl-Typen (2)

3. Hat der `unsigned` Typ einen Rang der mindestens gleich hoch ist wie der Rang des anderen Operanden, wird der andere Operand in den `unsigned` Typ umgewandelt.
   - Beispiel:

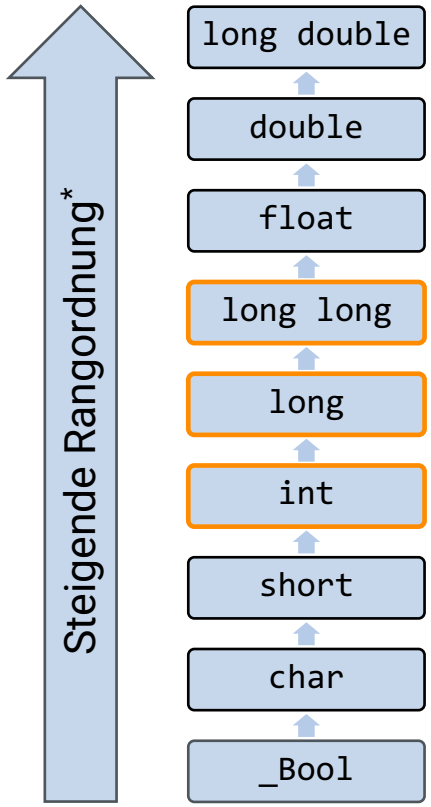
```
int a = -1;
unsigned long b = 2;
```

   Ausdruck: `a + b`
   → `a` wird in `unsigned long` umgewandelt (siehe Ganzzahl-Typ → vorzeichenloser Ganzzahl-Typ)
   Ergebnis wäre (wie erwartet) 1

4. Kann der `signed` Typ den gesamten Wertebereich des Typs des anderen Operanden darstellen, wird der andere Operand in den `signed` Typ umgewandelt.
   - Beispiel:

```
long long a = 5000LL;
unsigned b = 10u;
```

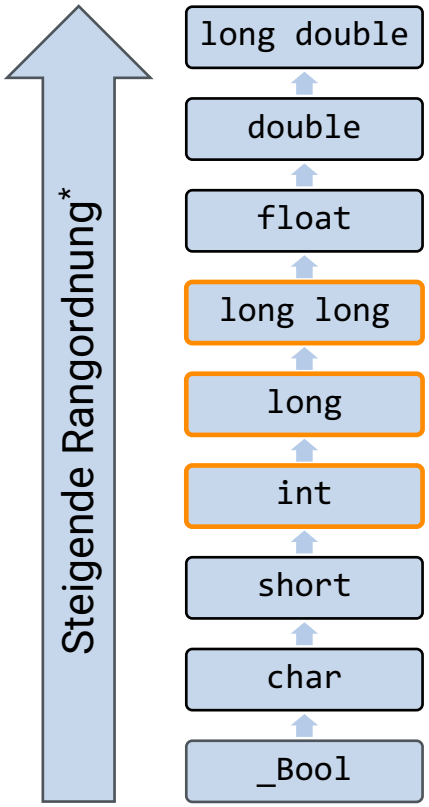
   Ausdruck: `a + b`
   → `b` wird in `long long` umgewandelt



* `signed` und `unsigned` Varianten desselben Ganzzahl-Typs haben dieselbe Rangordnung.

Gewöhnliche arithmetische Typumwandlung – Umwandlung bei Ganzzahl-Typen (3)

5. Andernfalls werden beide Operanden in den `unsigned` Typ mit demselben Rang des `signed` Typs umgewandelt.
   - untypisches Szenario
   - Tritt nur auf, wenn der `signed` Typ
     - höheren Rang als der `unsigned` Typ hat
     - nicht den gesamten Wertebereich des `unsigned` Typs abbilden kann
   - Beispiel:
     - `unsigned int` und `long` haben gleich viele Bits (C-Standard-konform, z.B. 32 Bit)
     - `long` kann somit nicht den gesamten Wertebereich von `unsigned int` abdecken → Umwandlung nach `unsigned long`
   - Kommt bei normalen C-Implementierungen nicht vor



* `signed` und `unsigned` Varianten desselben Ganzzahl-Typs haben dieselbe Rangordnung.

Aufgabe Datentyp der Ausdrücke

- Welchen Datentyp hat das Ergebnis der jeweiligen Ausdrücke?

Ausdruck	Datentyp des Ergebnisses
`2 + 5.0`	
`5.0f + 5`	
`1.0f + 2.0`	
`(int)10.0 + 5.0f`	
`2 + 3UL;`	
`'C' + '-' + 'V' + 'O'`	
`(char)(2 + 4)`	
`3 > 5`	
`(short)10 + 3U`	

Aufgabe Datentyp der Ausdrücke - Lösung

- Welchen Datentyp hat das Ergebnis der jeweiligen Ausdrücke?

Ausdruck	Datentyp des Ergebnisses
`2 + 5.0`	`double`
`5.0f + 5`	`float`
`1.0f + 2.0`	`double`
`(int)10.0 + 5.0f`	`float`
`2 + 3UL;`	`unsigned long`
`'C' + '-' + 'V' + 'O'`	`int`
`(char)(2 + 4)`	`char`
`3 > 5`	`int`
`(short)10 + 3U`	`unsigned`

Zusätzliche Übungsaufgaben

Aufgabe Operatoren (1)

- Gegeben sei eine Variable x vom Datentyp `unsigned`. Geben Sie einen Ausdruck an, um zu bestimmen, ob der Wert der Variable x gerade oder ungerade ist. Ist der Wert der Variable x gerade, soll der Ausdruck den Wert 1 haben, sonst den Wert 0. Lösen Sie diese Aufgabe ohne den Restwert-Operator (%)!
- Gegeben sei eine Variable x vom Datentyp `unsigned`. Geben Sie einen Ausdruck, der den Wert y und den Typ `unsigned` hat an, sodass $x \leq y \leq x + 1$ gilt und y ungerade ist.

Aufgabe Operatoren (2)

- Implementieren Sie ein Programm, welches alle Zweierpotenzen von 2^0 bis 2^n , d.h. $\{2^x : x \in \{0, 1, \dots, n\}\}$, für ein gewähltes n ausgibt. Versuchen Sie dieses Programm nur mit den folgenden Operatoren umzusetzen:
  - Zuweisungsoperator (=)
  - Zuweisungsoperator mit Shift (z.B. <<=)
  - Vergleichsoperatoren (z.B. <)
  - Shift Operatoren (z.B. <<)
  - Komma Operator (,)

Typumwandlungen

- Gehen Sie für die folgende Aufgabe davon aus, dass:
  - Negative Zahlen im Zweierkomplement dargestellt werden.
  - `sizeof(int)` den Wert 4 hat.
  - Ein Zeichensatz verwendet wird, welcher zum ASCII-Zeichensatz kompatibel ist.
- Bestimmen Sie den Wert und den Typ der folgenden Ausdrücke:
  - `4 >= 2 ? 5 : 3.4`
  - `'0' - '5' == -5`
  - `~0`
  - `(int)(2.5 * 3)`
  - `-10 + 1U + 10LL`